

try with resources :

There is a predefined interface available in java.io package called Closeable from JDK 1.5V onwards.

```
public interface Closeable
{
    void close() throws IOException;
}
```

* This interface was providing **manual closing** facility so, we need to call the close() method from finally block.
Example :

Scanner class implements from Closeable interface so, Scanner class need to override close method

```
Scanner sc = null;
try
{
    sc = new Scanner(System.in);
    //Reading the data
}
catch(InputMismatchException e)
{
}
finally
{
    sc.close(); //Manually we need to close the resource
}
```

* JDK 1.7V onwards, Java has provided a new interface called **java.lang.AutoCloseable** to provide automatic closing of all the resources which are implementing from AutoCloseable interface.

Example :
public interface AutoCloseable
{
 void close() throws Exception;
}

* In order to provide automatic closing facility through AutoCloseable, we need to implement two concepts :

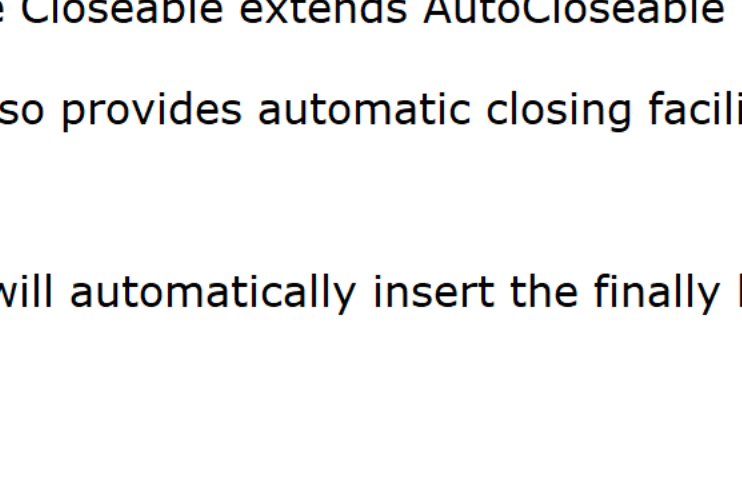
1) Any resource class (Predefined OR User-defined) **must implements** from AutoCloseable interface.

2) We need to pass the corresponding resource class object as a parameter to try block. [try()]

Example :

```
class Resource implements AutoCloseable
{
    //Must override close() method
}

Resource r1 = new Resource();
try(r1)
{
}
```



* Now java.io.Closeable interface extends from java.lang.AutoCloseable so, from JDK 1.7V onwards, Closeable interface is also providing automatic closing facility.

Example :
public interface Closeable extends AutoCloseable
{
 //Now It also provides automatic closing facility
}

* Java compiler will automatically insert the finally block when we use try with resources concept.

//Programs :

```
package com.ravi.try_with_resources;

import java.io.Closeable;
import java.io.IOException;

class DatabaseResource implements AutoCloseable
{
    @Override
    public void close() throws Exception
    {
        IO.println("Database resource closed successfully!!!");
    }
}

class FileResource implements Closeable
{
    @Override
    public void close() throws IOException
    {
        IO.println("File Resource closed successfully!!!");
    }
}

public class TryWithResourcesDemo1
{
    public static void main(String[] args) throws Exception
    {
        DatabaseResource dr = new DatabaseResource();
        FileResource fr = new FileResource();

        try(dr; fr)    //It will close right to left
        {
            IO.println(10/0);
        }
        catch(ArithmeticException e)
        {
            IO.println("Divide by zero problem");
        }
    }
}
```

```
package com.ravi.try_with_resources;

import java.util.InputMismatchException;
import java.util.Scanner;

public class TryWithResorsesDemo2
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        try(sc)
        {
            IO.println("Enter your Salary :");
            double salary = sc.nextDouble();
            IO.println("Your salary is :"+salary);
        }
        catch(InputMismatchException e)
        {
            IO.println("Input is not in a proper format");
        }
    }
}
```

**** Difference between Checked Exception and Unchecked Exception :

Strange behaviour of Java compiler :

- 1) Why compiler will generate exception at compile time
- 2) Sample.class file is already available but still javac -> ClassNotFoundException
- 3) 10/0; 100% AE but compiler will compile the code

Checked Exception :